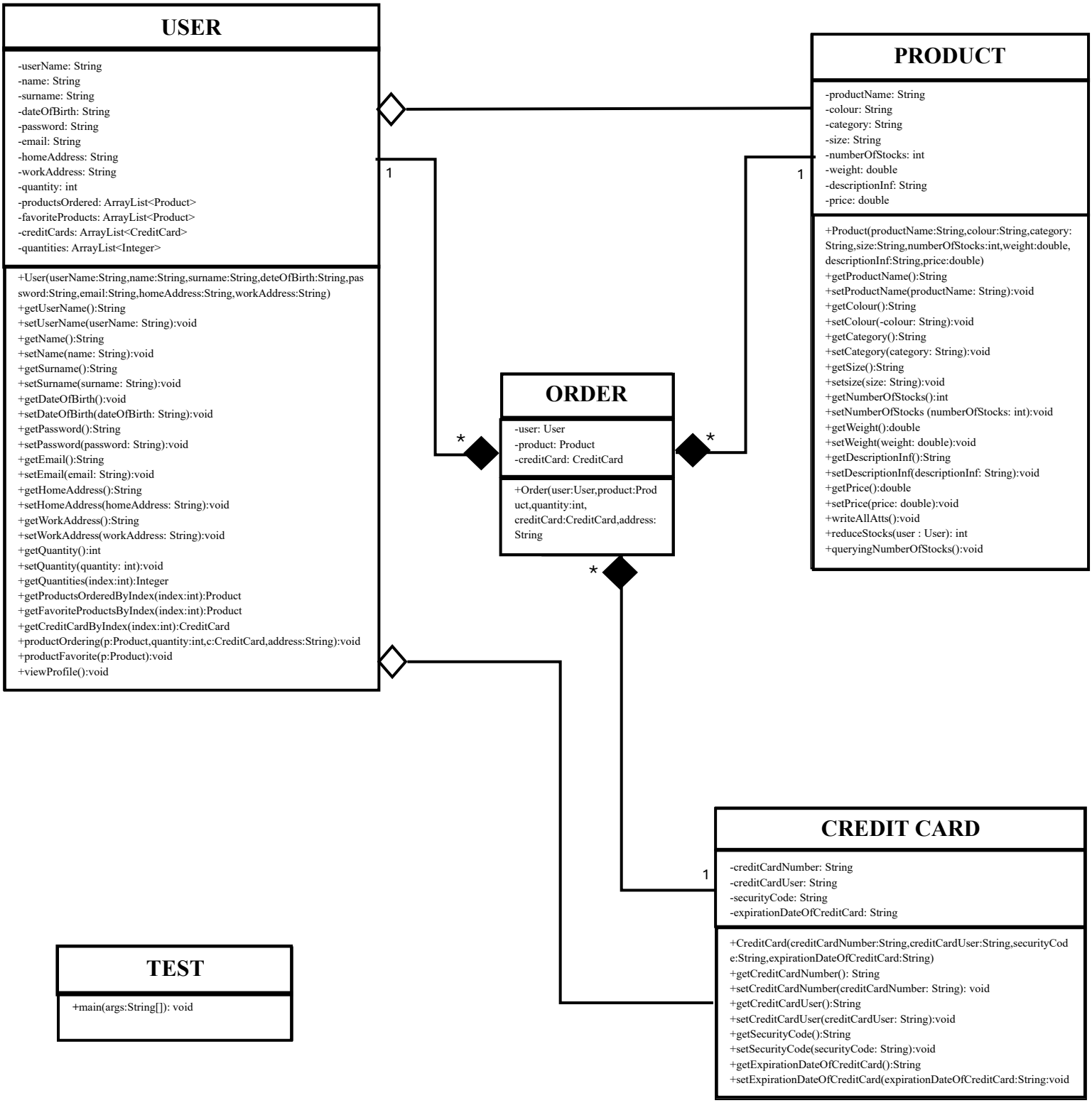
OBJECT ORIENTED PROGRAMMING PROJECT'S REPORT

E-COMMERCE SITE

13.06.2025

UML DIAGRAM


USER
-userName: String
-name: String
-surname: String
-dateOfBirth: String
-password: String
-email: String
-homeAddress: String
-workAddress: String
-quantity: int
-productsOrdered: ArrayList<Product>
-favoriteProducts: ArrayList<Product>
-creditCards: ArrayList<CreditCard>
-quantities: ArrayList<Integer>
+User(userName:String,name:String,surname:String,deteOfBirth:String,pas
sword:String,email:String,homeAddress:String,workAddress:String)
+getUserName():String
+setUserName(userName: String):void
+getName():String
+setName(name: String):void
+getSurname():String
+setSurname(surname: String):void
+getDateOfBirth():void
+setDateOfBirth(dateOfBirth: String):void
+getPassword():String
+setPassword(password: String):void
+getEmail():String
+setEmail(email: String):void
+getHomeAddress():String
+setHomeAddress(homeAddress: String):void
+getWorkAddress():String
+setWorkAddress(workAddress: String):void
+getQuantity():int
+setQuantity(quantity: int):void
+getQuantities(index:int):Integer
+getProductsOrderedByIndex(index:int):Product
+getFavoriteProductsByIndex(index:int):Product
+getCreditCardByIndex(index:int):CreditCard
+productOrdering(p:Product,quantity:int,c:CreditCard,address:String):void
+productFavorite(p:Product):void
+viewProfile():void
PRODUCT
-productName: String
-colour: String
-category: String
-size: String
-numberOfStocks: int
-weight: double
-descriptionInf: String
-price: double
+Product(productName:String,colour:String,category:
String,size:String,numberOfStocks:int,weight:double,
descriptionInf:String,price:double)
+getProductName():String
+setProductName(productName: String):void
+getColour():String
+setColour(-colour: String):void
+getCategory():String
+setCategory(category: String):void
+getSize():String
+setsize(size: String):void
+getNumberOfStocks():int
+setNumberOfStocks (numberOfStocks: int):void
+getWeight():double
+setWeight(weight: double):void
+getDescriptionInf():String
+setDescriptionInf(descriptionInf: String):void
+getPrice():double
+setPrice(price: double):void
+writeAllAtts():void
+reduceStocks(user : User): int
+queryingNumberOfStocks():void
ORDER
-user: User
-product: Product
-creditCard: CreditCard
+Order(user:User,product:Prod
uct,quantity:int,
creditCard:CreditCard,address:
String
CREDIT CARD
-creditCardNumber: String
-creditCardUser: String
-securityCode: String
-expirationDateOfCreditCard: String
+CreditCard(creditCardNumber:String,creditCardUser:String,securityCod
e:String,expirationDateOfCreditCard:String)
+getCreditCardNumber(): String
+setCreditCardNumber(creditCardNumber: String): void
+getCreditCardUser():String
+setCreditCardUser(creditCardUser: String):void
+getSecurityCode():String
+setSecurityCode(securityCode: String):void
+getExpirationDateOfCreditCard():String
+setExpirationDateOfCreditCard(expirationDateOfCreditCard:String:void
TEST
+main(args:String[]): void
1
1
*
*
*
1

User Class

Point of This Class:

This class was created to define users' information and to write the methods for the operations they will perform.

CLASS STRUCTURE AND MEMBERS:

1.VARIABLES AND ARRAY LISTS:

We have 9 variables and 4 array lists for this class:

Variables:

1. `userName` this variable holds users' user names.
2. `name` this variable holds users' names.
3. `Surname` this variable holds users' surnames.
4. `dateOfBirth` this variable holds users' date of births.
5. `password` this variable holds users' passwords.
6. `email` this variable holds users' e-mail adresses.
7. `homeAdress` this variable holds users' home adresses.
8. `workAdress` this variable holds users' work adresses.
9. `quantity` this variable holds the quantity of products that users ordered.

ArrayLists:

1. `ArrayList<Product> productsOrdered` this array list holds the product objects users ordered.
2. `ArrayList<Product> favoriteProducts` this array list holds users' favorite products.
3. `ArrayList<CreditCard> creditCard` this array list holds the credit cards users use.
4. `ArrayList<Integer> quantities` this array list holds the quantities of ordered products.

2.CONSTRUCTOR:

The constructor in this class is used to initialize the user's variables when a new user object is created.

```java
public User(String userName, String name, String surname, String dateOfBirth, String password, String email, String homeAddress, String workAddress){
    this.userName = userName;
    this.name = name;
    this.surname = surname;
    this.dateOfBirth = dateOfBirth;
    this.password = password;
    this.email = email;
    this.homeAddress = homeAddress;
    this.workAddress = workAddress;
}
```

3.GETTER AND SETTERS:

Getters and setters were added to allow controlled access to the private attributes of the class, following the principles of encapsulation.

```java
public String getUserName() {
    return userName;
}

public void setUserName(String userName) {
    this.userName = userName;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getSurname() {
    return surname;
}

public void setSurname(String surname) {
    this.surname = surname;
}
```

```java
public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getSurname() {
    return surname;
}

public void setSurname(String surname) {
    this.surname = surname;
}

public String getDateOfBirth() {
    return dateOfBirth;
}

public void setDateOfBirth(String dateOfBirth) {
    this.dateOfBirth = dateOfBirth;
}
```

```java
public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getHomeAddress() {
    return homeAddress;
}

public void setHomeAddress(String homeAddress) {
    this.homeAddress = homeAddress;
}
```

```java
public String getWorkAddress() {
    return workAddress;
}

public void setWorkAddress(String workAddress) {
    this.workAddress = workAddress;
}

public int getQuantity() {
    return quantity;
}

public void setQuantity(int quantity) {
    this.quantity = quantity;
}
```

4.OTHER METHODS:

getQuantities

```java
public Integer getQuantities(int index){
    return this.quantities.get(index);
}
```

This method accesses the `quantities` list and returns the value (as an `Integer`) corresponding to the given index. It helps retrieve how many units of a specific product the user has ordered.

getProductsOrderedByIndex

```java
public Product getProductsOrderedByIndex(int index){
    return this.productsOrdered.get(index);
}
```

This method returns a product from the user's list of ordered products based on the specified index.

getFavoriteProductsByIndex

```java
public Product getFavoriteProductsByIndex(int index){
    return this.favoriteProducts.get(index);
}
```

This method returns a product from the user's list of favorite products by using the given index.

getCreditCardByIndex

```java
public CreditCard getCreditCardByIndex(int index){
    return this.creditCards.get(index);
}
```

This method returns a credit card from the user's list of credit cards based on the provided index.

productOrdering

```java
public void productOrdering(Product p, int quantity, CreditCard c, String address){
    this.quantity= quantity;
    if((quantity<=p.getNumberOfStocks()) && (quantity>0)){

    productsOrdered.add(p);

    quantities.add(quantity);

    if(!creditCards.contains(c)){
        creditCards.add(c);
    }
        System.out.println("\n\n-------------------------------");
    System.out.println("\nYour Order Informations: ");
    System.out.println(p.getProductName() + " x" + quantity + "   " + p.getPrice()*quantity + "TL\n" );
    System.out.println("Credit Card Number: ************" + c.getCreditCardNumber().substring(12));
    System.out.println("Adress Information For This Order: " + address);
        System.out.println("\n-----------------------------\n\n");
    }
}
```

The `productOrdering` method is used to process a user's product order. It takes a Product, the ordered quantity, a CreditCard, and an address as input parameters. First, it checks whether the ordered quantity is positive and does not exceed the available stock of the product. If the condition is met, it adds the product to the productsOrdered list and the quantity to the quantities list. If the credit card used in the order is not already stored, it is added to the creditCards list. Finally, the method prints a summary of the order, including the product name, quantity, total price, masked credit card number, and delivery address.

productFavorite

```java
public void productFavorite(Product p){
    if(!favoriteProducts.contains(p)){
        favoriteProducts.add(p);
    }
}
```

The `productFavorite` method allows a user to mark a product as a favorite. It checks whether the product is already in the `favoriteProducts` list. If not, it adds the product to the list. This prevents duplicate entries and ensures each product appears only once in the user's list of favorites.

viewProfile

```java
public void viewProfile(){
    System.out.println("\n\n"+getName()+" "+getSurname()+"'s Account\n");
    System.out.println("Your favorite products: ");
    for(Product p : favoriteProducts){
        System.out.println(p.getProductName()+" ");
    }
    System.out.println();
    System.out.println("Your ordered products: ");
    for(int i = 0; i < productsOrdered.size(); i++){
        Product p = getProductsOrderedByIndex(i);
        int q = getQuantities(i);
        System.out.println(p.getProductName() + " x" + q);
    }
    System.out.println();
    System.out.println("Your credit cards: ");
    for(CreditCard c : creditCard){
        System.out.println(c.getCreditCardNumber() + " ");
    }
}
```

The `viewProfile` method displays the user's profile information, including their favorite products, ordered products with quantities, and registered credit cards. It first prints the list of favorite product names, then loops through the ordered products

and displays each product's name along with the quantity ordered. Finally, it prints the credit card numbers stored for the user. This method helps users review the key details associated with their account in a clear and organized format.

Product Class

Point of This Class:

This class was created to define products' information and hold the methods to be implemented regarding products.

CLASS STRUCTURE AND MEMBERS:

1.VARIABLES:

We have 8 variables for this class:

Variables:

1. `productName` this variable holds products' names.
2. `colour` this variable holds products' colours.
3. `category` this variable holds products' categories.
4. `size` this variable holds products' sizes.
5. `numberOfStocks` this variable holds products' numbers of stocks.
6. `weight` this variable holds products' weights.
7. `descriptionInf` this variable holds products' description informations.
8. `price` this variable holds products' prices.

2.CONSTRUCTOR:

The constructor in this class is used to initialize the product's variables when a new product object is created. The writeAllAtts method is called to print the product's properties as soon as the product is created.

```java
public Product(String productName, String colour, String category, String size, int numberOfStocks,double weight, String descriptionInf, double price){
    this.productName = productName;
    this.colour = colour;
    this.category = category;
    this.size = size;
    this.numberOfStocks = numberOfStocks;
    this.weight = weight;
    this.descriptionInf = descriptionInf;
    this.price = price;
    writeAllAtts();
}
```

3.GETTERS AND SETTERS:

To make private data fields in the Product class accessible, getters that return the value of these data fields and setters that update them were added, following the principles of encapsulation.

```java
public String getProductName() {
    return productName;
}

public String getColour() {
    return colour;
}

public String getCategory() {
    return category;
}

public String getSize() {
    return size;
}

public int getNumberOfStocks() {
    return numberOfStocks;
}
public double getWeight() {
    return weight;
}

public String getDescriptionInf() {
    return descriptionInf;
}

public double getPrice(){
    return price;
}

public void setProductName(String productName) {
    this.productName = productName;
}

public void setColour(String colour) {
    this.colour = colour;
}

public void setCategory(String category) {
    this.category = category;
}

public void setSize(String size) {
    this.size = size;
}

public void setNumberOfStocks(int numberOfStocks) {
    this.numberOfStocks = numberOfStocks;
}

public void setWeight(double weight) {
    this.weight = weight;
}

public void setDescriptionInf(String descriptionInf) {
    this.descriptionInf = descriptionInf;
}

public void setPrice(double price){
    this.price = price;
}
```

4.OTHER METHODS:

writeAllAtts

```
public void writeAllAtts(){
    System.out.println("Product Name: " + getProductName());
    System.out.println("Colour: " + getColour());
    System.out.println("Category: " + getCategory());
    System.out.println("Weight: " + getWeight());
    System.out.println("Size: " + getSize());
    System.out.println("Description Information: " + getDescriptionInf());
    System.out.println("Price: " + getPrice() + " TL");
    System.out.println("\n\n");
   }
```

The aim of this method is to print variables of the created products to the output. Whenever a new product is created in the Test class, this method is called by the constructor to print the properties of that product.

reduceStocks

```
public int reduceStocks(User user){
    if(getNumberOfStocks()>= user.getQuantity()){
        setNumberOfStocks(getNumberOfStocks() - user.getQuantity());
    }
    else{
        System.out.println("There is not enough stock. This order cannot be fulfilled.\n");
    }
        return getNumberOfStocks();
}
```

This method was created to ensure that the requested quantity is deducted from the stock if there is sufficient stock during the product order, but to print this situation on the output if there is not enough stock. When this method is called in the Order class, the correct quantity in the User class is deducted from the stock via the user parameter in the method.

queryingNumberOfStocks

```
public void queryingNumberOfStocks(){
    System.out.println();
    System.out.println(getProductName()+" has "+ getNumberOfStocks()+" stocks now");

}
```

This method was written to query the stock number of a product. Through this method, it is verified whether the reduceStock method works correctly.

CreditCard Class

Point of This Class:

This class was created to define credit card information and to write methods for the transactions they will perform.

CLASS STRUCTURE AND MEMBERS:

1.VARIABLES:

We have 4 variables for this class:

Variables:

1. `creditCardNumber` : This variable holds the 16-digit number located on the front of the credit card.
2. `creditCardUser` : This variable holds the name-surname of the cartholder written on the front of the card.
3. `securityCode` : This variable holds the 3-digit security code located on the back of the card.
4. `expirationDateOfCreditCard` : This variable holds the expiration date of the credit card located on the back of the credit card.

2.CONSTRUCTOR:

The constructor initializes the variables when a new instance of the CreditCard class is created.

```
public CreditCard (String creditCardNumber, String creditCardUser, String securityCode, String expirationDateOfCreditCard){
 this.creditCardNumber=creditCardNumber;
 this.creditCardUser=creditCardUser;
 this.securityCode = securityCode;
 this.expirationDateOfCreditCard=expirationDateOfCreditCard;
}
```

3.GETTER AND SETTERS

-The getter methods allow access to the value of a private variable form other class.

-The setter methods allow modification of the value of a private variable from outside the class.

-These methods are used to implement encapsulation.

*For variable creditCardNumber

```
public String getCreditCardNumber() {
    return creditCardNumber;
}

public void setCreditCardNumber(String creditCardNumber) {
    this.creditCardNumber = creditCardNumber;
}
```

*For variable creditCardUser

```
public String getCreditCardUser() {
    return creditCardUser;
}

public void setCreditCardUser(String creditCardUser) {
    this.creditCardUser = creditCardUser;
}
```

*For variable securityCode

```
public String getSecurityCode() {
    return securityCode;
}

public void setSecurityCode(String securityCode) {
    this.securityCode = securityCode;
}
```

*For variable expirationDateOfCreditCard

```
public String getExpirationDateOfCreditCard() {
    return expirationDateOfCreditCard;
}

public void setExpirationDateOfCreditCard(String expirationDateOfCreditCard) {
    this.expirationDateOfCreditCard = expirationDateOfCreditCard;
}
```

4.OTHER METHODS:

This class only contains getter and setter methods. Therefore, there are no other methods.

Order Class

Point of This Class:

This method was created to keep information about a user's shopping. It includes who made the order (the user), which products they bought and how much all the products cost in total. This helps us understand and manage what users buy in the system.

CLASS STRUCTURE AND MEMBERS:

1.VARIABLES:

We have 3 variables for this class:

1. `user` this variable holds the user who will place the order.
2. `product` this variable holds the product to be ordered.
3. `creditCard` this variable holds the credit card that will be used for the order.

2.CONSTRUCTOR:

```
public Order(User user, Product product, int quantity, CreditCard creditCard, String address){
    this.product = product;
    this.user = user;
    this.creditCard = creditCard;
    System.out.println("\n\nDo you want to complete ordering?");
    Scanner scan = new Scanner(System.in);
    String answer = scan.nextLine();
    if(answer.equalsIgnoreCase("YES")){
        user.productOrdering(product, quantity, creditCard, address);
        product.reduceStocks(user);
    }
    else{
        System.out.println("Your order has been cancelled.\n");
    }
}
```

This constructor method creates a new order. It takes the user, the product, how many products, a credit card, and the delivery address. Then it asks the user if they want to finish the order. If the answer is 'yes', it orders the product and reduces the product's stock. If not, it cancels the order.

Test Class

Point of This Class:

This class was created to check if the codes in other classes are working properly.

CLASS STRUCTURE AND MEMBERS:

1.TEST USERS;

```java
public class Test{

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        User u1 = new User("feyzaozdemir", "Feyza", "Ozdemir", "12.03.2005", "feyza123", "feyzaozdemir@email.com", "Efeler/Aydin", "Yunusemre/Manisa");
        User u2 = new User("meldagungor", "Melda", "Gungor", "27.03.2004", "melda06", "meldagungor06@email.com", "Mamak/Ankara", "Yunusemre/Manisa");
        User u3 = new User("kubratemur", "Kubra", "Temur", "04.05.2005", "kubra123", "kubratemur@email.com", "Arnavutkoy/Istanbul", "Yunusemre/Manisa");
```

2.TEST CREDİT CARDS:

```java
CreditCard c1 = new CreditCard("1111111111111111", "Feyza Ozdemir","123", "12/36");
CreditCard c2 = new CreditCard("2222222222222222", "Melda Gungor", "244", "12/30");
CreditCard c3 = new CreditCard("3333333333333333", "Kubra Temur",  "968", "09/32");
CreditCard c4 = new CreditCard("4444444444444444", "Melda Gungor", "777", "05/29");
```

3.TEST PRODUCTS:

```java
Product p1 = new Product("Old Season T-Shirt", "Yellow", "Clothing", "S", 10, 5.1, "It has a soft fabric and is oversize.", 699.99);
Product p2 = new Product("New Season T-Shirt", "Yellow", "Clothing", "M", 10, 6.1, "It has a stiff fabric and wraps the body.", 599.99);
Product p3 = new Product("Satchel bag","Brown","Bag","",30,100.0,"It is very useful for school",400.1);
Product p4 = new Product("High heels shoes","Black","Shoe","38",20,200.2,"It is recommended to buy one size larger.",1500.0);
Product p5 = new Product("The Little Prince book","","Book&Stationery&Hobby","",5,50.0,"This book is not only for children.",200.0);
Product p6= new Product ("Victoria's Secret bare vanilla body mist","","Cosmetics","",55,100.8,"Every particle deserves to be celebrated.",1350.0);
Product p7 = new Product("Lenovo gaming laptop","","Electronics","",1,2200.0,"LENOVO LOQ AMD Ryzen 5-7235HS Gaming Laptop",25999.9);
Product p8 = new Product("Philips Avent Baby Bottle","","Mother&Baby","",8,55.0,"It makes your baby's feeding experience more comfortable",489.99);
```

4. TEST METHOD INVOCATIONS:

1. After the products are entered and the code is executed, all products are visible in the code output.

```
Product Name: Old Season T-Shirt
Colour: Yellow
Category: Clothing
Weight: 5.1
Size: S
Description Information: It has a soft fabric and is oversize.
Price: 699.99 TL


Product Name: New Season T-Shirt
Colour: Yellow
Category: Clothing
Weight: 6.1
Size: M
Description Information: It has a stiff fabric and wraps the body.
Price: 599.99 TL


Product Name: Satchel bag
Colour: Brown
Category: Bag
Weight: 100.0
Size:
Description Information: It is very useful for school
Price: 400.1 TL


Product Name: High heels shoes
Colour: Black
Category: Shoe
Weight: 200.2
Size: 38
Description Information: It is recommended to buy one size larger.
Price: 1500.0 TL


Product Name: The Little Prince book
Colour:
Category: Book&Stationery&Hobby
Weight: 50.0
Size:
Description Information: This book is not only for children.
Price: 200.0 TL


Product Name: Victoria's Secret bare vanilla body mist
Colour:
Category: Cosmetics
Weight: 100.8
Size:
Description Information: Every particle deserves to be celebrated.
Price: 1350.0 TL


Product Name: Lenovo gaming laptop
Colour:
Category: Electronics
Weight: 2200.0
Size:
Description Information: LENOVO LOQ AMD Ryzen 5-7235HS Gaming Laptop
Price: 25999.9 TL


Product Name: Philips Avent Baby Bottle
Colour:
Category: Mother&Baby
Weight: 55.0
Size:
Description Information: It makes your baby's feeding experience more comfortable
Price: 489.99 TL
```

2. In test class

```
u1.productFavorite(p1);
```

User u1 added product p1 to their favorites. We will display this in the user's profile later.

3. In test class

```
Order o1 = new Order(u1,p1,7,c1,u1.getHomeAddress());
```

User u1 is ordering product p1. The code will ask the user whether they want to order the product. If they say yes, the order will be successfully completed. If they say no, the ordering process will be halted.

if it is assumed that the user said yes;

In output

```
Do you want to complete ordering
yes


-------------------------------

Your Order Informations:
Old Season T-Shirt x7   4899.93TL

Credit Card Number: ************1111
Adress Information For This Order: Efeler/Aydin

------------------------------
```

It calculates the total amount of the order in the code output as 699.99*7 and displays the correct result on the screen.

4. In test class

```
p1.queryingNumberOfStocks();
```

With this method, the stock of product p1 is queried.

We should observe a decrease in stock in the code output because it was purchased from product p1 beforehand.

In output

```
Old Season T-Shirt has 3 stocks now
```

Product p1 had 10 stocks, and since 7 were purchased, there are 3 stocks left.

5. In test class

```
Order o2 = new Order(u3,p1,2,c3,u3.getWorkAddress());
```

User u3 is ordering product p1. The code will ask the user whether they want to order the product. If they say yes, the order will be successfully completed. If they say no, the ordering process will be halted.

if it is assumed that the user said yes;

In output

```
Do you want to complete ordering
yes


-------------------------------

Your Order Informations:
Old Season T-Shirt x2   1399.98TL

Credit Card Number: ************3333
Adress Information For This Order: Yunusemre/Manisa

-------------------------------
```

It calculates the total amount of the order in the code output as 699.99*2 and displays the correct result on the screen.

6. In test class

```
p1.queryingNumberOfStocks();
```

With this method, the stock of product p1 is queried.

We should observe a decrease in stock in the code output because it was purchased from product p1 beforehand.

In output

```
Old Season T-Shirt has 1 stocks now
```

There were 3 stocks left earlier. Since 2 units of product p1 were taken a little while ago, it is seen from the code output that 1 unit of the product is left.

7. In test class

```
u3.productFavorite(p6);
```

User u3 added product p6 to their favorites. We will display this in the user's profile later

8. In test class

```
Order o3 = new Order(u2,p1,3,c2,u2.getWorkAddress());
```

User u2 is ordering product p1. The code will ask the user whether they want to order the product. If they say yes, the order will be successfully completed. If they say no, the ordering process will be halted.

if it is assumed that the user said no;

In output

```
Do you want to complete ordering
no
Your order has been cancelled.
```

The order cannot be processed because the user has decided to cancel it.

9. In test class

```
p1.queryingNumberOfStocks();
```

With this method, the stock of product p1 is queried.

The current stock count of product p1 was 1. Since the previous purchase transaction was not completed, it should again write 1 in the code output.

In output

```
Old Season T-Shirt has 1 stocks now
```

10. In test class

```
u3.productFavorite(p7);
```

User u3 added product p7 to their favorites. We will display this in the user's profile later.

11. In test class

```
u2.productFavorite(p5);
```

User u2 added product p5 to their favorites. We will display this in the user's profile later.

12. In test class

```
Order o4 = new Order(u2,p3,4,c4,u2.getHomeAddress());
```

User u2 is ordering product p3. The code will ask the user whether they want to order the product. If they say yes, the order will be successfully completed. If they say no, the ordering process will be halted.

if it is assumed that the user said yes;

In output

```
Do you want to complete ordering
yes


-------------------------------

Your Order Informations:
Satchel bag x4   1600.4TL

Credit Card Number: ************4444
Adress Information For This Order: Mamak/Ankara

------------------------------
```

It calculates the total amount of the order in the code output as 400.1*4 and displays the correct result on the screen.

13. In test class

```
p3.queryingNumberOfStocks();
```

With this method, the stock of product p3 is queried.

We should observe a decrease in stock in the code output because it was purchased from product p3 beforehand.

In output

```
Satchel bag has 26 stocks now
```

The stock number was initially 30, but after 4 were purchased, it became 26. The updated stock is displayed in the code output.

14. In test class

```
u3.productFavorite(p2);
```

User u3 added product p2 to their favorites. We will display this in the user's profile later.

15. In test class

```
Order o5= new Order(u1,p5,1,c1,u1.getWorkAddress());
```

User u1 is ordering product p5. The code will ask the user whether they want to order the product. If they say yes, the order will be successfully completed. If they say no, the ordering process will be halted.

if it is assumed that the user said yes;

In output

```
Do you want to complete ordering
yes


-------------------------------

Your Order Informations:
The Little Prince book x1   200.0TL

Credit Card Number: ************1111
Adress Information For This Order: Yunusemre/Manisa

------------------------------
```

It calculates the total amount of the order in the code output as 200.0*1 and displays the correct result on the screen.

16. In test class

```
u1.productFavorite(p2);
```

User u1 added product p2 to their favorites. We will display this in the user's profile later.

17. In test class

```
Order o6= new Order(u3,p5,2,c2,u3.getWorkAddress());
```

User u3 is ordering product p5. The code will ask the user whether they want to order the product. If they say yes, the order will be successfully completed. If they say no, the ordering process will be halted.

if it is assumed that the user said yes;

In output

```
Do you want to complete ordering
yes


-------------------------------

Your Order Informations:
The Little Prince book x2   400.0TL

Credit Card Number: ************2222
Adress Information For This Order: Yunusemre/Manisa

------------------------------
```

It calculates the total amount of the order in the code output as 200.0*2 and displays the correct result on the screen.

18. In test class

```
p8.queryingNumberOfStocks();
```

With this method, the stock of product p8 is queried.

In output

```
Philips Avent Baby Bottle has 8 stocks now
```

The initial stock of product p8 was 8. Since it hasn't been purchased yet, it is still 8. Therefore, in the code output, the stock number is seen as 8.

19. In test class

```
Order o7= new Order(u2,p6,2,c3,u2.getHomeAddress());
```

User u2 is ordering product p6. The code will ask the user whether they want to order the product. If they say yes, the order will be successfully completed. If they say no, the ordering process will be halted.

if it is assumed that the user said yes;

In output

```
Do you want to complete ordering
yes


-------------------------------

Your Order Informations:
Victoria's Secret bare vanilla body mist x2   2700.0TL

Credit Card Number: ************3333
Adress Information For This Order: Mamak/Ankara

-------------------------------
```

It calculates the total amount of the order in the code output as 1350.0*2 and displays the correct result on the screen.

20. In test class

```
Order o8= new Order(u1,p7,2,c1,u1.getHomeAddress());
```

User u1 is ordering product p7. The code will ask the user whether they want to order the product. If they say yes, the order will be successfully completed. If they say no, the ordering process will be halted.

if it is assumed that the user said yes;

In output

```
Do you want to complete ordering
yes

There is not enough stock. This order cannot be fulfilled.
```

This time, the code output indicated that there is insufficient stock because the current stock count of the product is less than what the user wants to order.

21. In test class

```
u3.productFavorite(p6);
```

Normally, with this code, product p6 should be added to the favorite list of user u3, but since user u3 has already added this product to their favorites beforehand, it will not be added again. Thus, the product name will not be written twice in the user profile.

22. In test class

```
u1.viewProfile();
```

With this code, the products purchased by the user u1, the favorite product list, and the credit cards they use will be displayed.

In output

```
Feyza Ozdemir's Account

Your favorite products:
Old Season T-Shirt
New Season T-Shirt

Your ordered products:
Old Season T-Shirt x7
The Little Prince book x1

Your credit cards:
111111111111111
```

23. In test class

```
u2.viewProfile();
```

With this code, the products purchased by the user u2, the favorite product list, and the credit cards they use will be displayed.

In output

```
Melda Gungor's Account

Your favorite products:
The Little Prince book

Your ordered products:
Satchel bag x4
Victoria's Secret bare vanilla body mist x2

Your credit cards:
4444444444444444
3333333333333333
```

The user u2 had two cards in their name, but one of the two cards shown in the profile belongs to her while the other belongs to someone else, because the user canceled her first purchase with the card belonging to her and the credit card was not saved for this reason. She made her second purchase with her own credit card. For the third purchase, she used a card that does not belong to her. In this credit card section, the cards shown are not the ones belonging to them but the cards they used for purchases.

24. In test class

```
u3.viewProfile();
```

With this code, the products purchased by the user u3, the favorite product list, and the credit cards they use will be displayed.

In output

```
Kubra Temur's Account

Your favorite products:
Victoria's Secret bare vanilla body mist
Lenovo gaming laptop
New Season T-Shirt

Your ordered products:
Old Season T-Shirt x2
The Little Prince book x2

Your credit cards:
3333333333333333
2222222222222222
BUILD SUCCESSFUL (total time: 18 seconds)
```

The code was written twice to add the p6 product to favorites, but it was mentioned that it would only be seen once in the output. Above, the p6 (Victoria's Secret bare vanilla body mist) product is seen to be listed only once in the favorite products list.

The user only has one card, but there are 2 credit cards on the credit card list. The reason for this is that the user completed her first purchase using someone else's card and completed her second purchase with her own card. The credit card list keeps track of the cards that were ordered, not the cards that belong to the user.